Modeling & Verification

Max Tschaikowski

Aalborg University, Denmark
(Based on slides of Mirco Tribastone)

Modeling and Analysis of mutual Exclusion Algorithms

Overview

- CCS model of an mutual exclusion algorithm
- Verification using HML with recursion
- Verification using behavioral equivalences
- Check with CAAL

Peterson's Exclusion Algorithm[1]

while true do
  noncritical section
  $b_i := \textbf{true}$
  $k := j$
  while $b_j \wedge k = j$ **do**
    skip
  end while
  critical section
  $b_i := \textbf{false}$
end while

- $i, j \in \{1, 2\}$ (process ids)
- b_1, b_2, k are shared variables
- $b_i = \textbf{true}$ means that process i is trying to enter the critical section
- k is the id of the process in the critical section
- Initially, $b_1 := \textbf{false}$ and $b_2 := \textbf{false}$
- The initial value of k is left unspecified

[1] *"The original solution due to Dekker is discussed at length by Dijkstra in [1]. Of the many reformulations given since, perhaps the best appears in [3]. Unfortunately the authors believe their correct solution is incorrect."* From G.L. Peterson, Myths About the Mutual Exclusion Problem, *Information Processing Letters*, **12**(3), 115–116, 1981.

CCS Model of Peterson's Algorithm

1. Identify the collection of *communicating systems* and their channels
   - Obviously, process 1 and 2
   - Also, the shared variables can be seen as *passive agents* that react to actions performed by the processes
   - Processes communicate with variables through read and write operations
   - Processes do not communicate with each other explicitly
2. Describe the behaviour of each agent
3. Compose agents through parallel composition and restriction

Communicating Boolean Variables

- For each process i there is a boolean variable b_i
- b_i has two *local states* (i.e., **true** and **false**)

$$B_{1f} \stackrel{def}{=} \overline{b1rf}.B_{1f} + b1wf.B_{1f} + b1wt.B_{1t} \ ,$$
$$B_{1t} \stackrel{def}{=} \overline{b1rt}.B_{1t} + b1wf.B_{1f} + b1wt.B_{1t} \ .$$

Similarly,

$$B_{2f} \stackrel{def}{=} \overline{b2rf}.B_{2f} + b2wf.B_{2f} + b2wt.B_{2t} \ ,$$
$$B_{2t} \stackrel{def}{=} \overline{b2rt}.B_{2t} + b2wf.B_{2f} + b2wt.B_{2t} \ ,$$

where the pattern for the channel name is $b\langle i\rangle\langle x\rangle\langle y\rangle$, with

- $i \in \{1, 2\}$ the process id
- $x \in \{r, w\}$ the kind of operation
- $y \in \{f, t\}$ the variable value to be written or read

Model of the *turn* Variable k

In the case of a protocol with only two concurrent processes, k may only take values 1 and 2, respectively denoted by K_1 and K_2 in

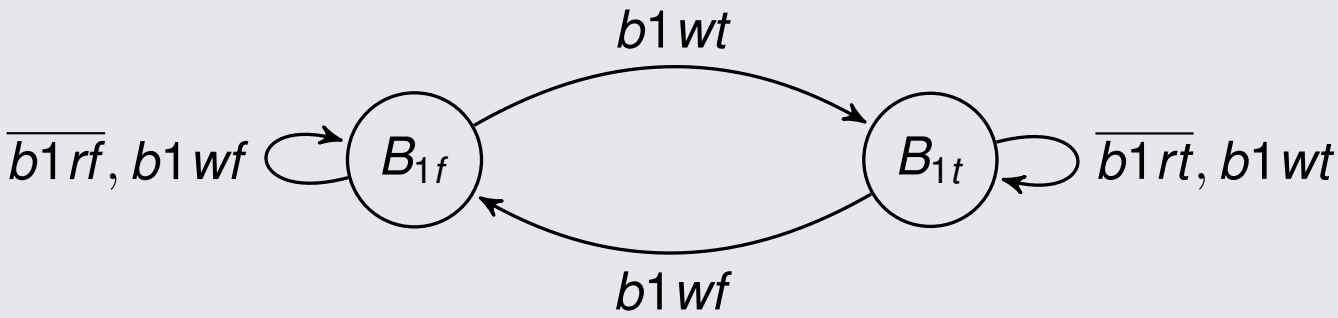
$$K_1 \stackrel{def}{=} \overline{kr1}.K_1 + kw1.K_1 + kw2.K_2 \ ,$$
$$K_2 \stackrel{def}{=} \overline{kr2}.K_2 + kw1.K_1 + kw2.K_2 \ ,$$

where the pattern for the channel name is $k\langle x\rangle\langle n\rangle$, with

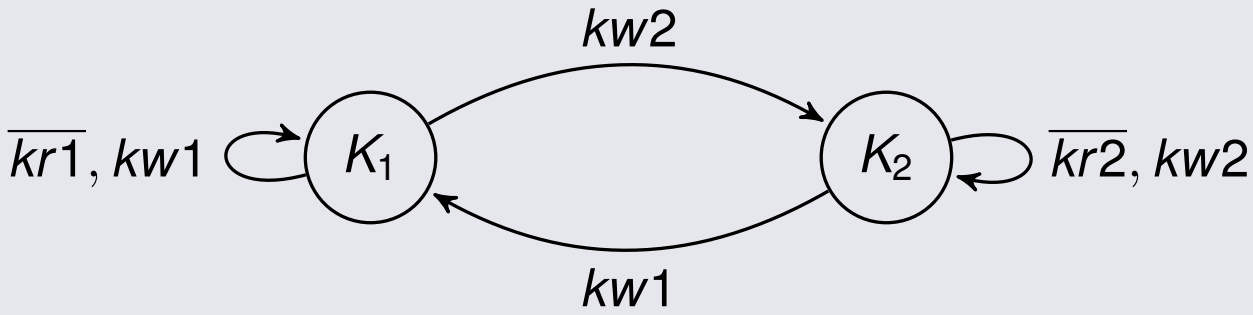
- $x \in \{r, w\}$ the kind of operation
- $n \in \{1, 2\}$ the value to be written or read

Labelled Transition Systems So Far

LTS of B_{1f} (B_{2f} is similar)



LTS of K_1



Model of Process 1

while true do
  noncritititical section
  $b_i := \textbf{true}$
  $k := j$
  while $b_j \wedge k = j$ **do**
    skip
  end while
  critical section
  $b_i := \textbf{false}$
end while

- Abstraction: we only focus on the entering and exiting of the critical section, omitting in particular the computation tasks of P_1, P_2.
- The process tries to enter:

$$P_1 \stackrel{def}{=} \overline{b1wt}.\overline{kw2}.P_{11}$$

- P_{11} models the **while** loop (with short-circuit evaluation):

$$P_{11} \stackrel{def}{=} b2rf.P_{12} + b2rt.(kr2.P_{11} + kr1.P_{12})$$

- P_{12} models the critical section:

$$P_{12} \stackrel{def}{=} enter_1.exit_1.\overline{b1wf}.P_1$$

Process 1 and Process 2

Process 1

$$P_1 \stackrel{def}{=} \overline{b1wt}.\overline{kw2}.P_{11}$$

$$P_{11} \stackrel{def}{=} b2rf.P_{12} + b2rt.(kr2.P_{11} + kr1.P_{12})$$

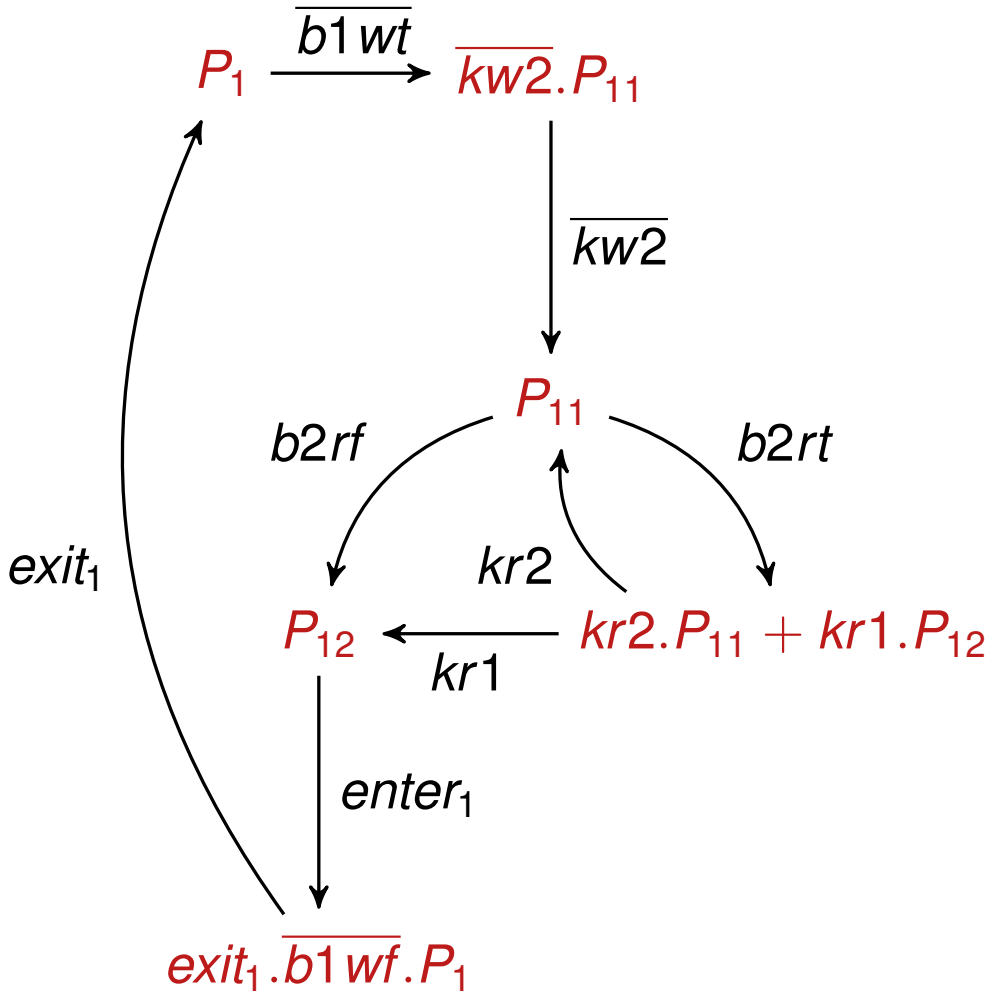
$$P_{12} \stackrel{def}{=} enter_1.exit_1.\overline{b1wf}.P_1$$

Process 2

$$P_2 \stackrel{def}{=} \overline{b2wt}.\overline{kw1}.P_{21}$$

$$P_{21} \stackrel{def}{=} b1rf.P_{22} + b1rt.(kr1.P_{21} + kr2.P_{22})$$

$$P_{22} \stackrel{def}{=} enter_2.exit_2.\overline{b2wf}.P_2$$



LTS of P_1

Peterson's System

$$B_{1f} \stackrel{def}{=} \overline{b1rf}.B_{1f} + b1wf.B_{1f} + b1wt.B_{1t}$$

$$B_{1t} \stackrel{def}{=} \overline{b1rt}.B_{1t} + b1wf.B_{1f} + b1wt.B_{1t}$$

$$B_{2f} \stackrel{def}{=} \overline{b2rf}.B_{2f} + b2wf.B_{2f} + b2wt.B_{2t}$$

$$B_{2t} \stackrel{def}{=} \overline{b2rt}.B_{2t} + b2wf.B_{2f} + b2wt.B_{2t}$$

$$K_1 \stackrel{def}{=} \overline{kr1}.K_1 + kw1.K_1 + kw2.K_2$$

$$K_2 \stackrel{def}{=} \overline{kr2}.K_2 + kw1.K_1 + kw2.K_2$$

$$P_1 \stackrel{def}{=} \overline{b1wt}.\overline{kw2}.P_{11}$$

$$P_{11} \stackrel{def}{=} b2rf.P_{12} + b2rt.(kr2.P_{11} + kr1.P_{12})$$

$$P_{12} \stackrel{def}{=} enter_1.exit_1.\overline{b1wf}.P_1$$

$$P_2 \stackrel{def}{=} \overline{b2wt}.\overline{kw1}.P_{21}$$

$$P_{21} \stackrel{def}{=} b1rf.P_{22} + b1rt.(kr1.P_{21} + kr2.P_{22})$$

$$P_{22} \stackrel{def}{=} enter_2.exit_2.\overline{b2wf}.P_2$$

$$Peterson \stackrel{def}{=} \big(B_{1f} \mid B_{2f} \mid K_1 \mid P_1 \mid P_2\big)\backslash L\,,$$

$$L = \{b1rf, b1rt, b1wf, b1wt, b2rf, b2rt, b2wf, b2wt, kr1, kw1, kr2, kw2\}$$

Verification of Peterson's Algorithm

Informal Verification Criterion

At no point in the execution of the algorithm processes P_1 and P_2 are in their critical sections at the same time

- $\text{Inv} \stackrel{\max}{=} F \wedge [\text{Act}]\text{Inv}$
- $F = [\text{exit}_1]\text{ff} \vee [\text{exit}_2]\text{ff}$

Verification via HML

After computing Inv, one obtains that $\mathit{Peterson} \in \text{Inv}$.

Verification via Behavioral Equivalences

Find a suitable specification process *Spec* and ...

- ... check if *Peterson* is bisimilar to *Spec*
- ... check if *Peterson* is trace equivalent to *Spec*

Verification with Behavioral Equivalences

Informal Verification Criterion

At no point in the execution of the algorithm processes P_1 and P_2 are in their critical sections at the same time

Using Strong Bisimulation?

$$MutexSpec \stackrel{def}{=} enter_1.exit_1.MutexSpec + enter_2.exit_2.MutexSpec\ .$$

$$\text{Is } MutexSpec \sim Peterson?$$

Using the game characterisation of strong bisimulation:

- Attacker says: $MutexSpec \xrightarrow{enter_1} exit_1.MutexSpec$
- Defender loses because no $enter_1$ action is enabled by $Peterson$:

$$Peterson \xrightarrow{\tau} (B_{1t} \mid B_{2f} \mid K_1 \mid \overline{kw2}.P_{11} \mid P_2)\backslash L\ , \text{and}$$
$$Peterson \xrightarrow{\tau} (B_{1f} \mid B_{2t} \mid K_1 \mid P_1 \mid \overline{kw1}.P_{21})\backslash L\ .$$

Using Weak Bisimulation?

$$MutexSpec \stackrel{def}{=} enter_1.exit_1.MutexSpec + enter_2.exit_2.MutexSpec\ .$$

Is $MutexSpec \approx Peterson$?

1. Attacker chooses right for a sufficient number of times to have
$$Peterson \stackrel{\tau}{\Rightarrow} (B_{1t} \mid B_{2t} \mid P_{12} \mid P_{21} \mid K_1)\backslash L\ ,$$
2. to which the defender responds by
$$MutexSpec \stackrel{\tau}{\Rightarrow} MutexSpec\ .$$
3. Now, the attacher chooses left and says
$$MutexSpec \xrightarrow{enter_2} exit_2.MutexSpec$$
4. but the defender does not afford any $enter_2$-transitions.

Using Trace Equivalence?

Property

Peterson is not trace equivalent to *MutexSpec*.

Weak Traces and Weak Trace Equivalence

A *weak trace* of a process P is a sequence $a_1 \cdots a_k$, $k \geq 1$, of observable actions such that there exists a sequence of transitions

$$P = P_0 \stackrel{a_1}{\Rightarrow} P_1 \stackrel{a_2}{\Rightarrow} \cdots \stackrel{a_k}{\Rightarrow} P_k \ ,$$

for some $P_1, \ldots, P_k$. Two processes are *weak trace equivalent* if the afford the same weak traces.

Property

Peterson is weak trace equivalent to *MutexSpec*.